

grading:

1	2	3	4	Σ
14	5	8	30	57

Practice exam for XB_0024 Multi-Agent Systems

This exam will test your knowledge and understanding of the material provided to you and presented in the lectures, the book of Blackburn, Bos and Striegnitz *Learn Prolog Now!* (Chapters 1 to 6, 10 and 11), and the MARBEL *Guide*. It is *not* allowed to use materials such as books, papers or slides during the exam. You will have **2:45 hours to complete the exam**. Your answers must be provided on the answer sheet only. It has 4 questions, for a total of 57 points. There are two types of questions: multiple choice questions and open questions. For multiple choice questions, you will see a table like this:

(a): option 1	(b): option 2	(c): option 3	(d): ...
---------------	---------------	---------------	----------

To answer, *circle* the option you believe is correct. You can correct a mistake by crossing out the mistaken answer and then circling the new one.

For open questions, you will see a box in which you are meant to write your answer. Please *stay within the box*. Text outside of the box *will not be graded*.

Before you hand in your exam sheet, please check that you have filled in the following:

Full name and surname:

Student number:

Signature: _____

Date: _____

Questions

Question 1

14 points

Prolog Basics

3 points

(a) Give Prolog's answer for each of the following queries:

Query	Your answer
?-X=4+3.	
?-c(A,g(a)) = c(b,g(B)).	
?-\+(N=5).	
?- t(A,A) = t(B,C).	
?- X+2 is 5.	

4 points

(b) For the following question, assume the following Prolog base:

```
man(dilbert).  
man(bill).  
husband(bill).
```

Give Prolog's answer for each of the following queries:

Query	Answer
man(bill), not(husband(bill)).	
man(X), not(husband(X)).	
not(husband(dilbert)), man(dilbert).	
not(husband(X)), man(X).	

4 points

(c) Consider the following Prolog programme.

```
a(t).  
a(x).  
a(b(_)).  
b(y).  
b(t).  
b(_).  
c(t).  
c(x).
```

```
d(A,B) :- a(A),b(B),not(A=B).  
d(A,B) :- c(A),d(A,B).
```

Give the outcome of Prolog's search tree for each of the following queries (choose between **success**, **failure** and **infinite**):

Query	Answer
$d(b(b(y)),y).$	
$d(X,Y).$	
$d(t,t)$	
$d(b(y),b(y)).$	

3 points

(d) Consider the following predicate and query:

```
addFiveAndHalve(X,Y):- Y is (X+5)/2.  
  
?- addFiveAndHalve(X,3).
```

Give the outcome of the query, and explain why this is the case.

Question 2

5 points

Programming in Prolog

5 points

- (a) Given is a Prolog predicate `eNSum(L,N,S)` which succeeds if `S` is the sum of all elements in `L` up to and including the `N`th element.

```
eNSum([X|T],1,X).
eNSum([X|T],N,S):- N>1, NNew is N-1, eN(T,NNew,SNew), S is SNew + X.
```

The predicate gives the following query outputs:

```
?- eNSum([1,2,5,6],3,S).
S = 8.
?- eNSum([5],4,S).
false.
```

The predicate is written in such a way that there are still search queries on the stack when the base clause has been reached, which is called **left-recursion**. Rewrite the predicate to enable tail-recursion.

Question 3

8 points

Agents and MARBEL: Basics

4 points

- (a) Multi-agent environments can be classified according to different properties.

1. Explain what it means for an environment to be *deterministic/stochastic*.

2. Mention *two* further properties of environments.

Remark 1: Important for the MARBEL questions: When you add code in the agent templates, clearly specify the position in the code where you want to add something.

For example, if you want to *modify* some module file's line 11, use the following notation:

file.mod2g 11 ...put modified code here....

If you want to place code in some module file between two line numbers, e.g. 23 and 24, use the following notation:

file.mod2g 23.1 ...new code line 1....

file.mod2g 23.2 ...new code line 2....

file.mod2g 23.3 ...new code line 3....

<pre> 1 use "sensors-and-actuators.jar". 2 3 define securityRobot { 4 use securityInit for init. 5 use security as for decisions. 6 7 ... 8 ... 9 } 10 11 launchpolicy { 12 when * launch securityRobot. 13 } </pre>	<pre> 1 use security. 2 3 module securityInit { 4 ... 5 } </pre>
<pre> 1 :- dynamic alarm/0, at/1, safe/1. 2 3 zones(30). floor(1). floor(2). floor(3). 4 recharge :- ... </pre>	<pre> 1 use security. 2 3 module security { 4 % If robot detects it is not safe, sound the alarm. 5 if floor(X), not(safe(X)) then soundAlarm. 6 % If alarm is on but everything is safe, 7 %then silence the alarm. 8 if alarm then silenceAlarm. 9 % Move from floor 1, to 2, and then to 3; 10 % move back from floor 3 to 1. 11 if ... then scanFloor(Floor). 12 } </pre>

Figure 1: The securityRobot agent.

4 points

- (b) Suppose that an agent is connected to an entity that is already performing a durative action in the environment. What might happen when an agent requests the (entity in the) environment to perform an action while the current action it is performing has not yet finished? Mention at least two possibilities.

Question 4

30 points

Programming in MARBEL This assignment is about a (toy) security robot that is deployed in a parking garage with 3 floors with 30 parking zones. The robot is able to recharge only at floor 1 of the garage. The robot is always able to perform the scanFloor(Destination) for moving from one floor to the next. Carefully analyse the MARBEL agent system in Figure 1.

Follow the instructions given in Remark 1 in Figure 1.

5 points

- (a) The security robot is initially at floor 1. Add an action rule to the agent program in the right place to make sure that the query `at(1)` succeeds. Also add a rule to make sure that the queries of the form `safe(X)` succeed for all three floors. Indicate for both rules where they should be added.

5 points

- (b) Assume that `battery/1` is a *send on-change percept* informing the agent about its current battery level, which is also sent when the agent is just launched. Add a channel for handling this percept

with the appropriate handler to the code and indicate where you would add this. What else needs to be added to the program to make it work without problems (again indicate also where).

3 points

- (c) The `safe/1` is a *send always percept* informing the agent about whether it is safe at a floor. Add a channel for handling this percept with the appropriate handler to the code and indicate where you would add this.

2 points

- (d) The recharge level is set to 5. That is, if the battery level is less than 5 the robot should recharge its battery. The recharge predicate has not been defined yet in the `security.pl` Prolog file, see line 3. Provide a rule to define this predicate.

3 points

- (e) The `securityRobot` agent can always sound the alarm but needs to keep track of whether the alarm is on or not itself. Assume this action is instantaneous and the robot sounds the alarm when it performs this action. In other words, no percept is needed (and also not received) but the agent needs to keep track of whether the agent is sounding the alarm itself. Modify the program to make sure that the query `alarm` succeeds when the agent has just performed the `soundAlarm` action.

3 points

- (f) As the `securityRobot` agent needs to keep track itself of whether the alarm is on or not, the agent should update its database when it silences the alarm to make sure the query `alarm` does not succeed (any more). Modify the code to ensure this and indicate where changes need to be made.

2 points

- (g) The `securityRobot` agent should only silence the alarm if all floors are safe. Why do we not need to add something to the query of the action rule in line 8 of the `security.mod2g` file to check this? Explain this by reasoning about the other rules in this module.

4 points

- (h) The query in the condition of the action rule at line 11 in `security.mod2g` that selects the `scanFloor` action is missing. Add code to make sure the robot will move as specified in the comment directly above this rule. *Hint:* You may also want to add new Prolog code. If you do, indicate also where.

3 points

- (i) The agent never selects the **recharge** action but that is the most important thing to do when recharging is needed. Add a rule in the right place to make sure that the agent will recharge its battery when needed. Make sure to take the preconditions for this action into account in the query of the rule!

End of Exam